POLYTECHNIQUE MONTRÉAL
UNIVERSITÉ D'INGÉNIERIE

Département de génie informatique et de génie logiciel

LOG6951A

Stratégies d'exploitation de l'IA générative

TP2 : ResearchPal v2

Soumis à : M. Simon Barrette

Taha-Chahine HAMITOUCHE - 2118576

Felix Paille DOWELL - 2256243

19 avril 2026

Introduction:

L'application ResearchPal reposait initialement sur une architecture RAG simple. Bien qu'efficace dans les cas standards, elle présentait une limite importante : l'absence d'un mécanisme de correction ou de rétroaction. Ainsi, lorsqu'une erreur survenait lors de l'étape de retrieval (par exemple une mauvaise sélection de documents ou une requête mal formulée), cette erreur se propageait directement à la phase de génération, ce qui dégradait significativement la qualité des réponses produites.

C'est précisément cette faiblesse que nous avons cherché à adresser avec l'évolution de l'application. Plutôt que de repartir de zéro, nous avons fait le choix de conserver l'architecture de base de ResearchPal, à savoir les trois composantes principales : le retrieval, la génération, ainsi que l'amélioration de requête. Cette décision nous permet de capitaliser sur les éléments déjà performants du système tout en limitant les risques liés à une refonte complète.

À cette base, nous avons greffé une architecture de programmation agentique. Cette approche introduit une forme de raisonnement itératif et de prise de décision, permettant au système de s'auto-corriger, de réévaluer ses choix et d'adapter dynamiquement ses actions en fonction du contexte. Concrètement, cela se traduit par la capacité de détecter des réponses insuffisantes, de reformuler la requête, ou encore de relancer le processus de retrieval afin d'obtenir des informations plus pertinentes.

Cette hybridation entre une architecture RAG classique et une logique agentique reflète un scénario réaliste fréquemment observé en industrie. Elle permet de combiner la robustesse et la simplicité d'un pipeline structuré avec la flexibilité et l'adaptabilité d'un système plus intelligent et autonome.

Nous avons choisi de poursuivre le développement à partir de la codebase de Taha-Chahine, car elle intégrait déjà LangChain à toutes les étapes du pipeline. Cette base cohérente et modulaire a facilité l'ajout de nouvelles composantes, notamment la mise en place de LangGraph, sans nécessiter de refonte majeure.

Diagramme et architecture:

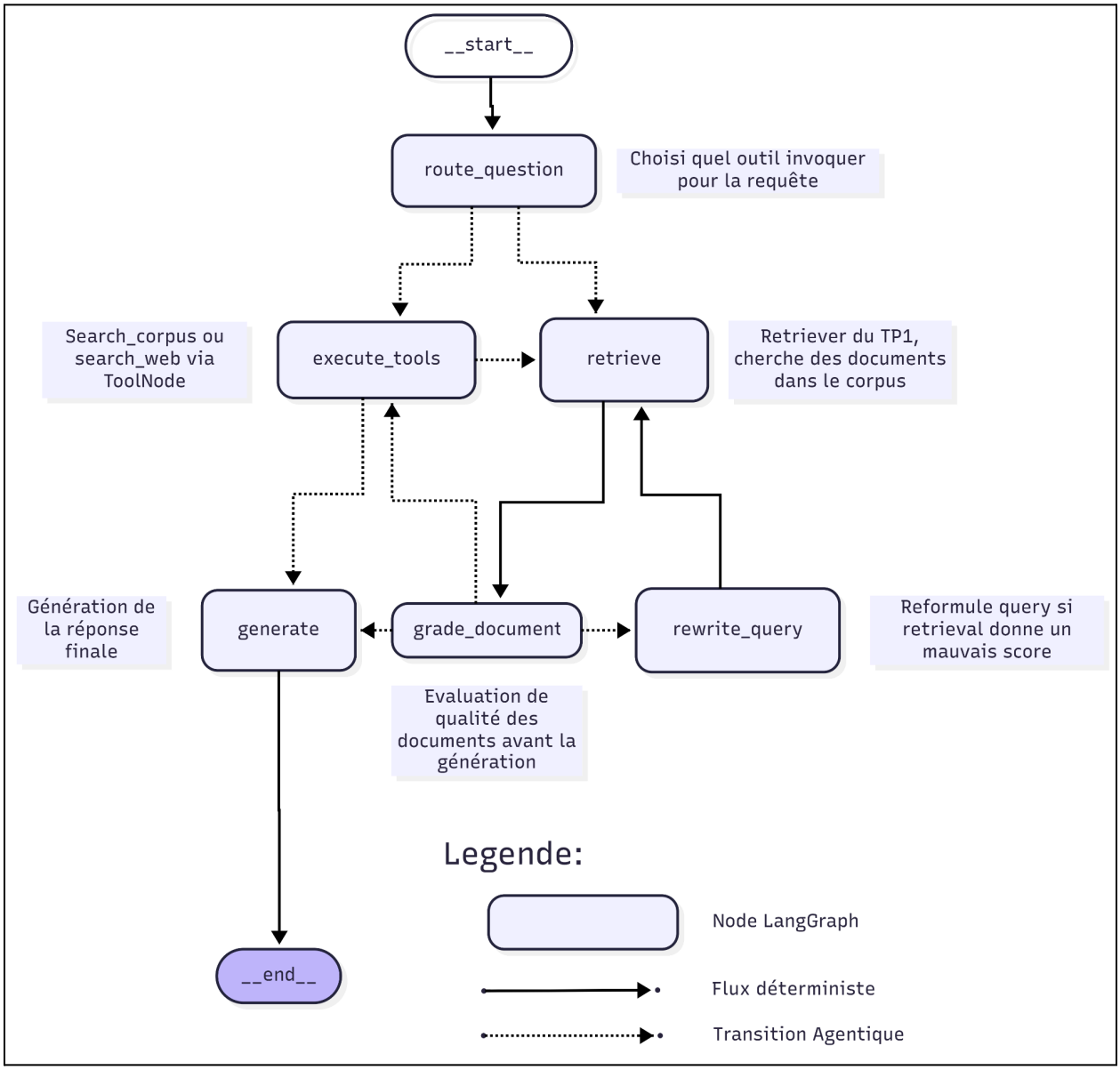


Figure 1 - Architecture LangGraph de *ResearchPal*

Le nouveau modèle peut initialement décider, à partir de la requête, s'il veut passer au retrieval classique ou invoquer une recherche de corpus ou en-ligne via ToolNode. S'il choisit d'invoquer une recherche corpus/web, il peut ensuite supplémenter avec un retrieval ou passer directement à la génération.

Le retrieval est supplémenté avec une étape d'évaluation, via la node grade_documents. Si le système détermine que le retrieval n'est pas assez bon, il reformule la query et exécute un nouveau retrieval. Après trois échecs de retrieval, il passe à la recherche web. Au cas contraire, il peut passer à la génération de la réponse.

Pattern de cycle:

Nous avons choisi d'adopter un pattern de *corrective RAG*, comme en témoigne le cycle formé entre les nœuds *retrieve*, *grade_documents* et *rewrite_query* dans la figure 1. Ce mécanisme introduit une étape explicite d'évaluation de la qualité de la récupération des documents. Ainsi, lorsque les documents récupérés sont jugés insuffisants ou peu pertinents, le système est capable de corriger la requête et de relancer le processus de récupération. Cela permet d'améliorer progressivement la pertinence du contexte fourni à la génération, tout en évitant de propager des erreurs en aval.

Ce choix architectural a également été motivé par des considérations de performance. En effet, nous utilisons Ollama, un LLM exécuté localement, dont les appels sont relativement coûteux en temps de calcul. Dans une approche de type *self-reflective RAG*, l'évaluation et la correction interviennent après la génération de la réponse, ce qui implique qu'un appel LLM potentiellement inutile ait déjà été effectué. À l'inverse, le *corrective RAG* permet d'effectuer cette évaluation en amont, avant la génération. Cela réduit les appels inutiles au modèle, optimise l'utilisation des ressources, et améliore l'efficacité globale du système, tout en conservant les avantages d'une architecture agentique capable de s'adapter dynamiquement.

Outils:

Le système a accès à deux outils principaux : search_corpus et search_web. L'outil *search_corpus* implémente un wrapper autour de ChromaDB, permettant au système d'effectuer des recherches vectorielles efficaces au sein du corpus de test. En complément, l'outil *search_web*, implémenté via l'interface de DuckDuckGo, permet d'élargir le champ de recherche lorsque l'information n'est pas présente

dans le corpus local. Cet outil retourne les résultats sous forme de document JSON, ce qui facilite leur traitement et leur intégration dans le pipeline RAG.

Le choix de DuckDuckGo repose principalement sur des considérations pratiques : il s'agit d'un service gratuit, ne nécessitant pas de clé API, ce qui simplifie son intégration tout en évitant les contraintes liées aux quotas ou à la gestion d'authentification. Ainsi, la combinaison de ces deux outils permet au système de trouver un équilibre entre fiabilité (corpus local) et couverture (recherche web).

Les descriptions d'outils ont été conçues pour guider efficacement le modèle dans leur utilisation, en réduisant les appels inutiles et les erreurs. Chaque outil précise clairement quand l'utiliser (cas pertinents), quand ne pas l'utiliser (situations à éviter) et le format de retour attendu, ce qui permet au système de mieux interpréter les résultats et de les intégrer correctement dans le pipeline.

Par exemple, pour *search_web* on demande d'utiliser l'outil quand la question porte sur des évènements récents qui ne sont pas couverts par le corpus. On demande de ne pas utiliser l'outil si la question est, par exemple, clairement contenue dans le corpus ou ne demande que des calculs ou de la logique. On demande aussi un retour JSON sérialisé avec cet forme: [{"title": "...", "href": "...", "body": "..."}, ...]. Ceci permet un parsing efficace des données trouvées.

Tableau 1 - Sélection dynamique d'outils selon le type de requête

Type de requête (Outil attendu)	Question	Outil utilisé	Latence (ms)
CORPUS	Record de buts de Alexander Ovechkin	corpus	100569
WEB	Résultats NHL récents	web	78701
MIXTE	Ovechkin actif en 2025 + stats corpus	corpus	110006

Les résultats illustrent que la sélection d'outils fonctionne bien quand la tâche est claire. Par exemple, la question sur les résultats récents de la LNH a bien lancé une recherche web, alors que la question sur le record de buts est passée par le corpus. Cependant, notre node n'implémente pas une stratégie pour les questions mixtes. Le résultat est alors une recherche au corpus seule.

Mémoire:

ResearchPal v2 implémente une mémoire long-terme épisodique légère. Cette approche consiste à stocker un nombre limité de résolutions de requêtes complexes réussies dans un fichier JSON persistant, puis à les réinjecter sous forme d'exemples *few-shot* dans le *system prompt* au début de chaque session. Cela permet à l'agent de s'appuyer sur des cas passés pertinents afin d'améliorer la qualité et la structure de ses réponses futures.

Ce choix est motivé par le cas d'usage de *ResearchPal v2*, qui agit comme un assistant de recherche. Dans ce contexte, les résolutions précédentes de questions complexes constituent des exemples particulièrement utiles pour guider le raisonnement du modèle. Contrairement à un cache sémantique, cette approche ne nécessite pas de calcul de similarité à chaque requête, ce qui est avantageux étant donné l'utilisation d'un LLM local (Ollama) avec un coût computationnel élevé. De plus, l'option des préférences utilisateur est moins pertinente ici, car l'application est centrée sur un corpus spécifique plutôt que sur une personnalisation utilisateur poussée.

Cependant, cette approche présente certaines limites. Afin de ne pas surcharger le *prompt*, le nombre d'exemples stockés est restreint (par exemple à 5 entrées), ce qui peut limiter la diversité des situations couvertes. De plus, la qualité des performances dépend fortement de la pertinence des exemples sélectionnés : des exemples mal choisis ou trop spécifiques peuvent biaiser les réponses du modèle. Enfin, cette mémoire reste statique entre les sessions et ne garantit pas une adaptation fine à toutes les variations de requêtes. Malgré ces limites, elle offre un bon compromis entre efficacité, coût computationnel et amélioration de la qualité des réponses.

Analyse des traces:

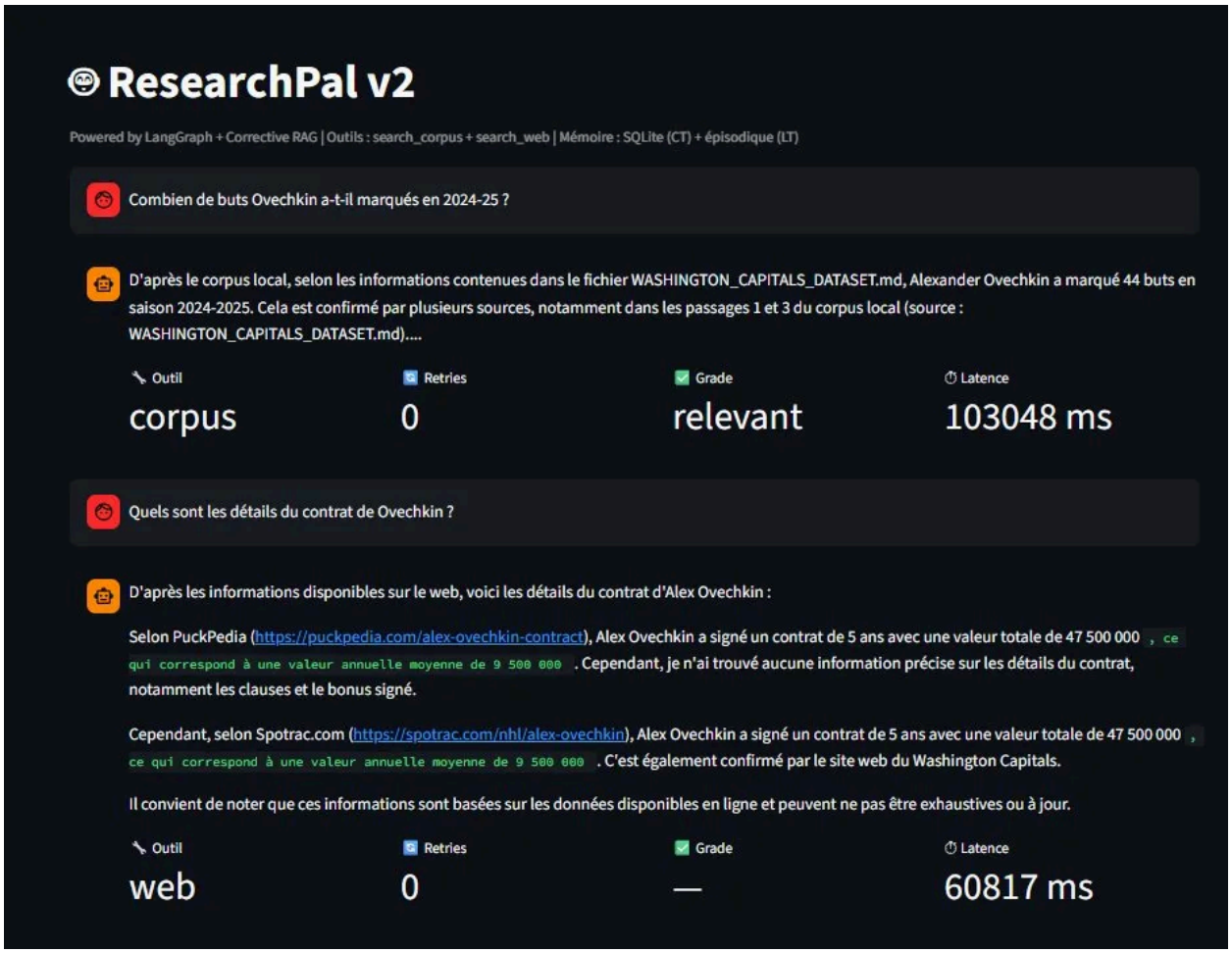


Figure 2 - Capture d'écran de la conversation instrumentée (Q1, Q2)

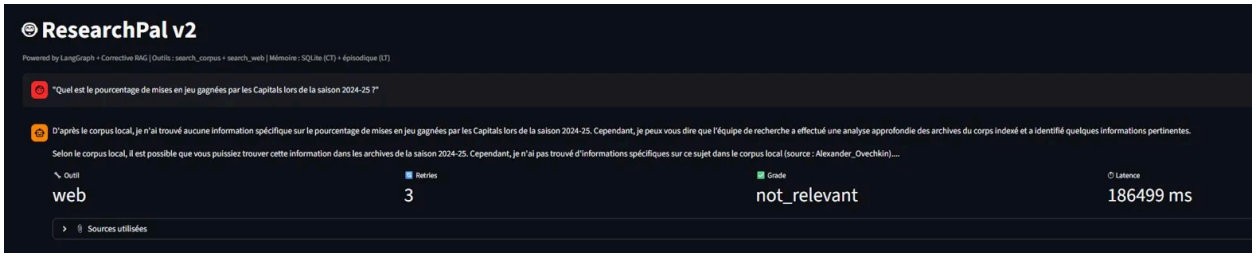


Figure 3 - Capture de la conversation instrumentée (Q4)

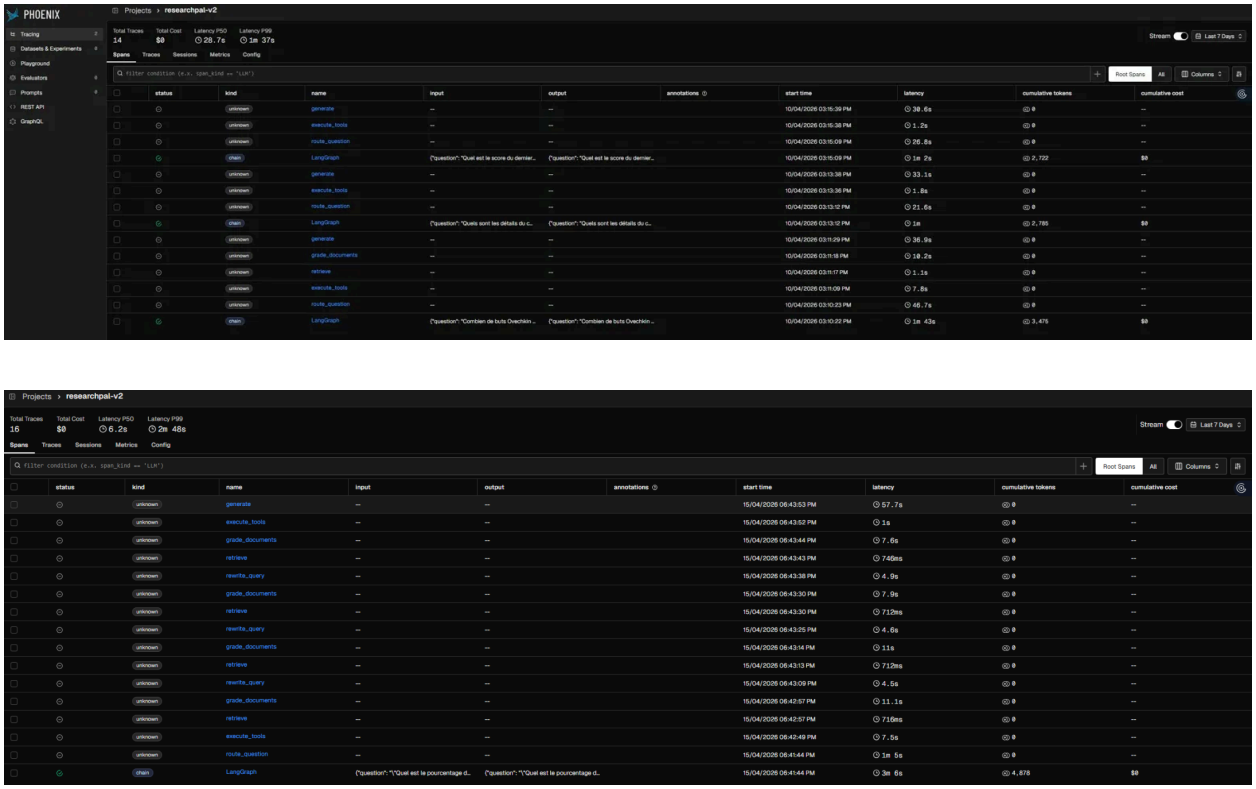


Figure 4 - Vue globale des traces

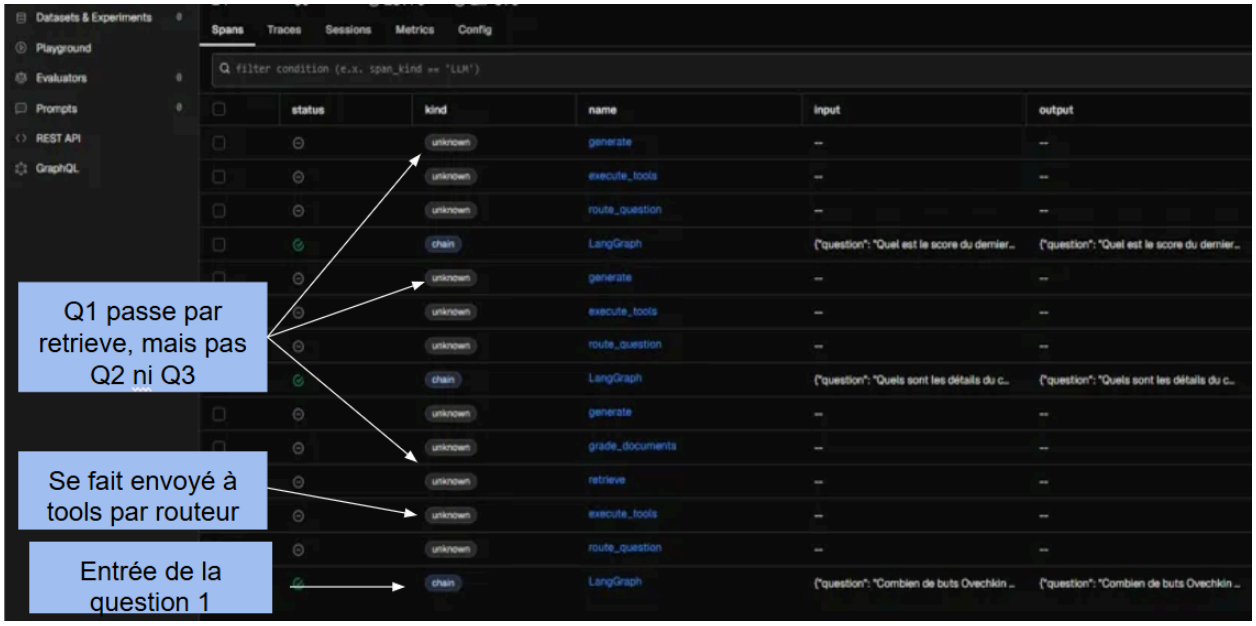


Figure 5 - Vue globale annotée

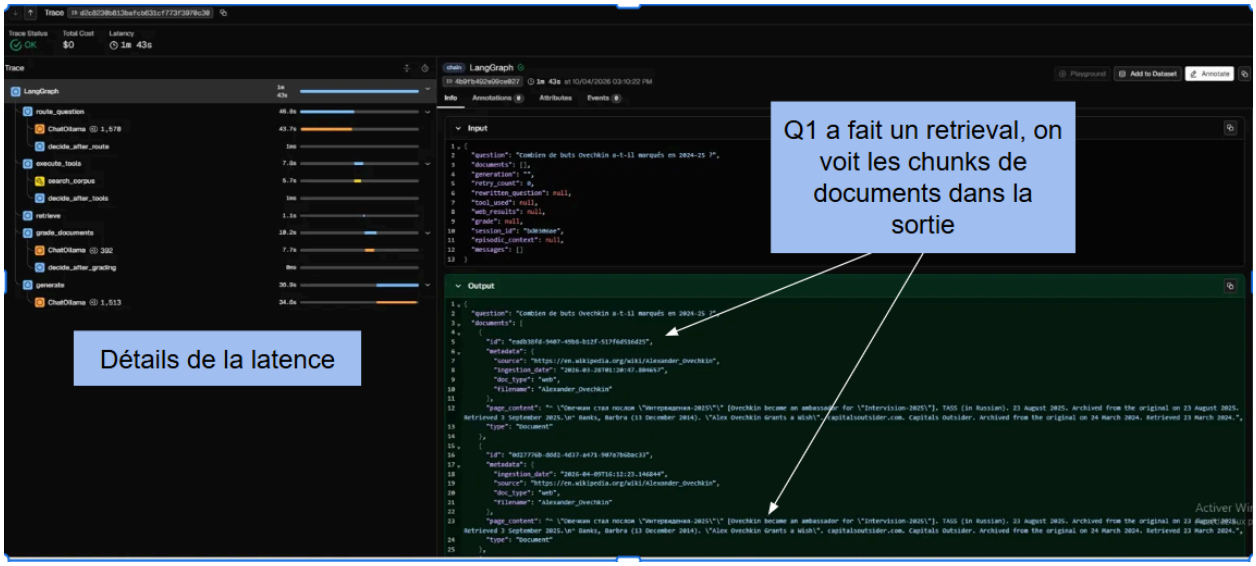


Figure 6 - Vue détaillée de Q1

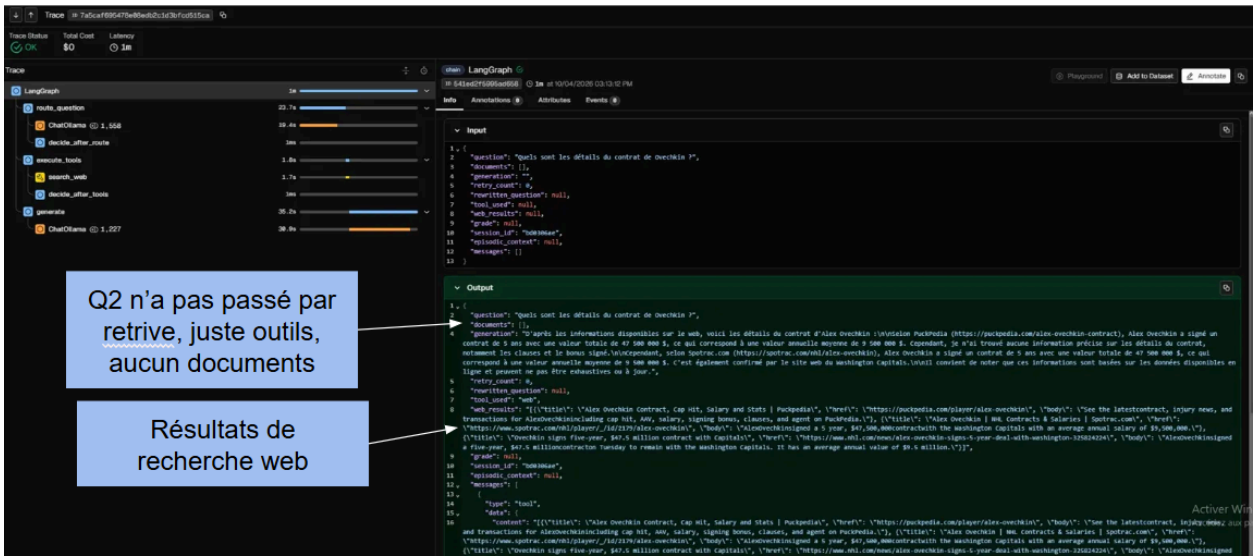
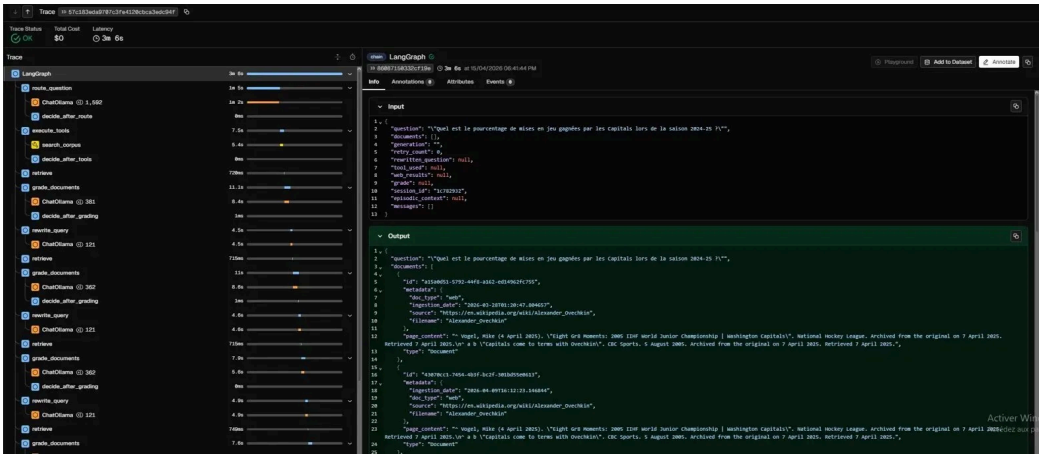


Figure 7 - Vue détaillée de Q2



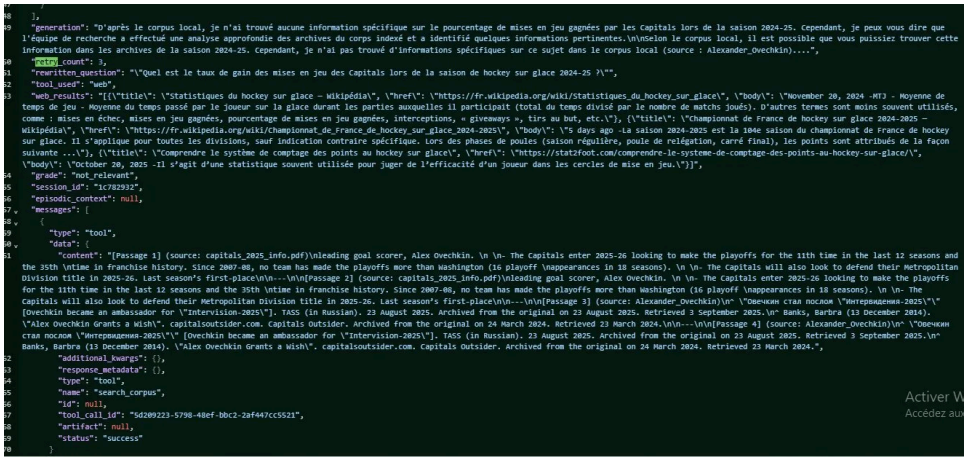


Figure 8 - Vue détaillée de Q4

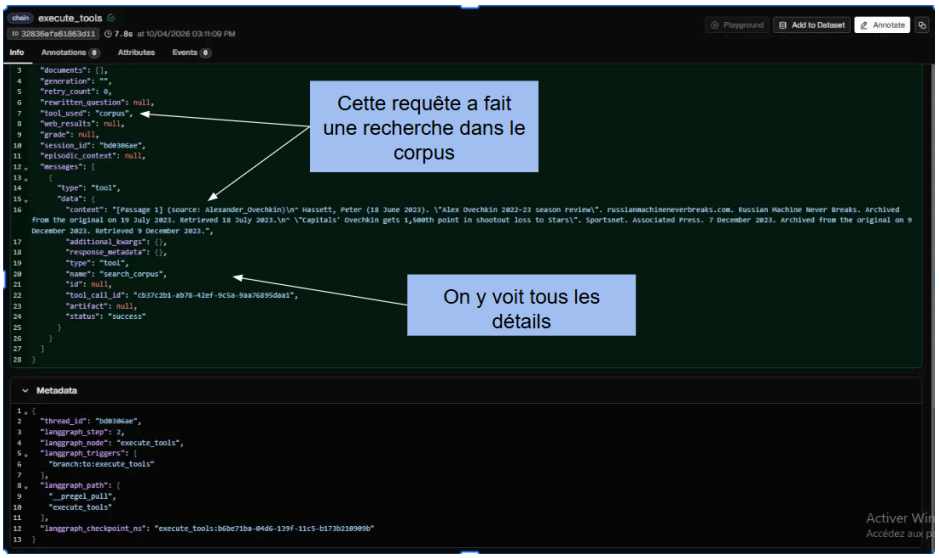


Figure 9 - Inspection de *execute_tools* sur une requête

Ici on analyse les traces de 3 questions posées au système. Par la figure 4, on peut voir qu'une seule de ces requêtes a effectué un retrieval classique, alors que les deux autres ont uniquement fait appel à un outil. Les figures 5, 6 et 7 permettent de comparer les chemins empruntés et de décomposer la latence par nœud.

Les étapes impliquant un appel LLM constituent clairement les goulots d'étranglement. Le tableau ci-dessous résume les métriques clés pour les 3 requêtes :

Tableau 2 - Décomposition de la latence par nœud

Métrique	Q1	Q2	Q3	Q4
Latence totale (ms)	103 048	60 817	62 860	186 499
Latence route_question (s)	46.8	21.6	26.8	65
Latence generate (s)	36.9	35.2	30.6	57.7
Tokens consommés	3 475	2 785	2 722	4 878
Itérations cycle correctif	0	0	0	3
Durée appel outil (s)	5.7	1.7	1.2	5.4 (search_corpus) + 1 (search_web_fallback)

Le nœud *route_question* représente à lui seul 45% de la latence totale sur Q1 (46.8s / 103s). Il effectue un appel LLM complet avec liaison d'outils (*bind_tools*) pour une simple décision de routing, dont le sous-span ChatOllama interne dure 43.7s, soit 94% du temps du nœud. Même la recherche web (1.7s) est négligeable en comparaison.

Conséquemment, une piste d'optimisation pertinente serait de remplacer ce nœud par un classifier léger basé sur des règles de mots-clés (ex. : « récent », « hier soir »

pour le web ; sinon voir le corpus). Cela permettrait de réserver llama3.2 exclusivement au nœud *generate*, ce qui permettrait de réduire la latence totale d'au moins 45% (le classifier étant pratiquement instantané).

De plus, afin de démontrer le fonctionnement du cycle correctif, la question « Quel est le pourcentage de mises en jeu gagnées par les Capitals lors de la saison 2024-25 ? » a été soumise au système, celle-ci étant absente du corpus. La trace Phoenix (Figure 8) montre le cycle *retrieve* → *grade_documents* → *rewrite_query* répété 4 fois avant d'atteindre le garde-fou MAX_RETRIES = 3. L'état final confirme retry_count: 3 et grade: "not_relevant", déclenchant le fallback vers search_web. La question reformulée était : « Quel est le taux de gain des mises en jeu des Capitals lors de la saison de hockey sur glace 2024-25 ? », montrant que le système est en mesure de s'auto-corriger et de basculer vers une source alternative lorsque le corpus est insuffisant.

Résultats RAGAS:

Le dataset d'évaluation utilisé comprend 15 paires question-réponse de référence réparties en trois catégories : 10 paires issues du corpus (C01 à C10, dont les réponses sont directement présentes dans les documents indexés sur les Washington Capitals et Ovechkin), 3 paires adversariales (A01 à A03, qui sont des questions hors corpus conçues pour provoquer des hallucinations en supposant des faits fictifs comme, par exemple, la retraite d'Ovechkin), et 2 paires multi-hop (M01 et M02, qui nécessitent la combinaison d'informations provenant de deux passages distincts).

Tableau 3 - Résultats RAGAS et LLM-as-judge

Métrique	Score
Faithfulness	*0.267*
Answer Relevance	*0.267*
Qualité des citations (LLM-as-judge)	*0.511*

Tableau 4 - Performance par type de question

Type de questions	Performance observée
Questions simples (corpus)	Bonne (réponses correctes et appuyées par les sources)
Questions complexes (corpus)	Faible (hallucinations ou réponses incomplètes)
Questions adversariales	Très faible (difficulté à refuser de répondre)
Questions multi-hop	Faible (manque de raisonnement et de combinaison d'informations)

Les résultats obtenus à l'aide de RAGAS montrent des performances globalement faibles pour les métriques principales. Le score de *faithfulness* (0.267) et celui de *answer relevance* (0.267) indiquent que le système produit fréquemment des réponses non supportées par le contexte ou qui ne répondent pas directement à la question posée. Il est à noter que l'évaluation RAGAS a été implémentée manuellement via des appels LLM séquentiels directs (voir **src/evaluation/ragas_eval.py**), plutôt qu'en utilisant le framework RAGAS officiel. Ce choix a été fait en raison des incompatibilités entre l'infrastructure asynchrone de RAGAS et Ollama, qui causaient des timeouts. Les métriques *faithfulness* et *answer_relevancy* sont ainsi calculées sur une échelle binaire (0 ou 1) par le LLM, puis moyennées sur les 15 paires, ce qui explique la granularité plus faible des scores obtenus. Les résultats que nous avons obtenus suggèrent que le modèle génère souvent du contenu en l'absence d'informations suffisantes dans les documents récupérés. Du côté du LLM-as-judge, le score de qualité des citations (0.511) est moyen, ce qui indique que les sources sont parfois présentes, mais qu'elles sont souvent imprécises, incomplètes ou mal utilisées. Le prompt d'évaluation de celui-ci est disponible dans **src/evaluation/llm_judge.py** (variables *SYSTEM_JUDGE_PROMPT* et *USER_JUDGE_PROMPT*). Il définit quatre critères : présence des citations, précision des sources, cohérence du format, et honnêteté (aveu d'ignorance). Le barème va de 0 (aucune citation) à 3 (sources précises et format cohérent), avec sortie JSON obligatoire.

Plusieurs facteurs expliquent ces faibles performances. Tout d'abord, le système présente des hallucinations fréquentes : lorsque les documents récupérés ne contiennent pas l'information demandée, le modèle génère malgré tout une réponse en inventant des faits, ce qui fait chuter la *faithfulness*. Ensuite, certaines réponses restent vagues ou générales et ne répondent pas directement à la question, ce qui impacte la *answer relevance*. De plus, la qualité du *retrieval* est variable : lorsque les passages récupérés sont incomplets ou peu pertinents, la génération repose sur un contexte inadéquat. Enfin, l'absence d'un mécanisme de refus robuste pousse le système à répondre même lorsqu'il ne dispose pas de l'information nécessaire, plutôt que d'indiquer explicitement ses limites.

Ces résultats illustrent que le cycle de *corrective RAG* n'est qu'une solution partielle. En permettant de réévaluer la pertinence des documents et de reformuler

la requête, il peut améliorer dans certains cas la qualité du contexte fourni à la génération, notamment pour des questions ambiguës ou mal formulées. Cependant, son impact reste limité, car il n'agit que sur la phase de récupération. Il ne permet pas d'empêcher les hallucinations au moment de la génération ni de garantir une utilisation correcte des sources. Ainsi, même avec un meilleur *retrieval*, le modèle peut produire des réponses incorrectes s'il ne dispose pas d'un mécanisme de vérification ou de refus plus strict.

Analyse des cas d'échec et des paires adversariales:

L'analyse des paires adversariales met particulièrement en évidence les limites du système. Face à des questions hors corpus ou piégeuses, le système ne parvient pas à adopter un comportement prudent. Au lieu de reconnaître l'absence d'information, il tente de générer une réponse, souvent incorrecte ou non pertinente. Dans plusieurs cas, aucune source fiable n'est fournie, ou les citations sont vagues. Ce comportement indique que le système privilégie la production d'une réponse, même erronée, plutôt que l'abstention, ce qui nuit à sa fiabilité.

Les questions nécessitant un raisonnement plus complexe, notamment les paires multi-hop, posent également problème. Le système a de la difficulté à combiner plusieurs passages pour construire une réponse cohérente, ce qui se traduit par des réponses incomplètes ou incorrectes. Les citations associées à ces réponses sont souvent absentes ou mal alignées avec le contenu utilisé. Dans l'ensemble, ces résultats montrent que le système fonctionne correctement pour des questions simples directement couvertes par le corpus, mais qu'il demeure fragile dès que l'information est absente, ambiguë ou nécessite une combinaison de plusieurs sources.

Utilisation IA Générative:

Le code de l'application a été en partie généré à l'aide du modèle Claude, sous supervision humaine afin de simuler le rôle d'ingénieur chef. Nous avons pris les décisions d'architecture. Par ailleurs, le modèle ChatGPT a été utilisé pour améliorer la qualité rédactionnelle du rapport, notamment en facilitant la reformulation, la structuration et la clarté des analyses présentées.

Conclusion:

Malgré les améliorations apportées par l'architecture agentique et le *corrective RAG*, le système présente encore plusieurs limites importantes. En particulier, il demeure vulnérable aux hallucinations lorsque l'information n'est pas présente dans le contexte. De plus, les performances restent faibles sur les questions complexes, notamment les cas multi-hop, où le système ne parvient pas à combiner efficacement plusieurs sources d'information. La qualité des citations est également variable, ce qui limite la fiabilité des réponses produites. Parmi les pistes d'évolution non implémentées, on peut envisager l'ajout d'un module de validation post-génération (*self-reflection* ou *verification step*) et l'introduction de contraintes plus strictes sur l'utilisation des sources. Enfin, l'intégration d'un mécanisme explicite de gestion de l'incertitude, permettant au système de refuser de répondre lorsque les informations sont insuffisantes, constituerait une amélioration significative de la robustesse globale.